adl-assignment1-group4

December 7, 2025

1 ADVANCED DEEP LEARNING Assignment 1 (Group 4)

SL.NO	BITS ID	NAME	CONTRIBUTION
1	2024AA05733	Bollam Manindra	100%
2	2024AA05734	S.Hari Chandana	100%
3	2024AA05732	Ashvini Satish Boshale	100%
4	2024AA05735	Nikhil Yadav	100%
5	2024AA05946	Deepthi Pullarkat	100%

[ ]:
```
# Install required packages
!pip install numpy
!pip install matplotlib
!pip install tensorflow
!pip install scikit-learn
```

```
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages
(2.0.2)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-
packages (3.10.0)
Requirement already satisfied: contourpy>=1.0.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-
packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (4.60.1)
Requirement already satisfied: kiwisolver>=1.3.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (1.4.9)
Requirement already satisfied: numpy>=1.23 in /usr/local/lib/python3.12/dist-
packages (from matplotlib) (2.0.2)
Requirement already satisfied: packaging>=20.0 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (25.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-
packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (3.2.5)
Requirement already satisfied: python-dateutil>=2.7 in
```

```
/usr/local/lib/python3.12/dist-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-
packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
Requirement already satisfied: tensorflow in /usr/local/lib/python3.12/dist-
packages (2.19.0)
Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.12/dist-
packages (from tensorflow) (1.4.0)
Requirement already satisfied: astunparse>=1.6.0 in
/usr/local/lib/python3.12/dist-packages (from tensorflow) (1.6.3)
Requirement already satisfied: flatbuffers>=24.3.25 in
/usr/local/lib/python3.12/dist-packages (from tensorflow) (25.9.23)
Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 in
/usr/local/lib/python3.12/dist-packages (from tensorflow) (0.6.0)
Requirement already satisfied: google-pasta>=0.1.1 in
/usr/local/lib/python3.12/dist-packages (from tensorflow) (0.2.0)
Requirement already satisfied: libclang>=13.0.0 in
/usr/local/lib/python3.12/dist-packages (from tensorflow) (18.1.1)
Requirement already satisfied: opt-einsum>=2.3.2 in
/usr/local/lib/python3.12/dist-packages (from tensorflow) (3.4.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-
packages (from tensorflow) (25.0)
Requirement already satisfied:
protobuf!=4.21.0,!=4.21.1,!=4.21.2,!=4.21.3,!=4.21.4,!=4.21.5,<6.0.0dev,>=3.20.3
in /usr/local/lib/python3.12/dist-packages (from tensorflow) (5.29.5)
Requirement already satisfied: requests<3,>=2.21.0 in
/usr/local/lib/python3.12/dist-packages (from tensorflow) (2.32.4)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-
packages (from tensorflow) (75.2.0)
Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.12/dist-
packages (from tensorflow) (1.17.0)
Requirement already satisfied: termcolor>=1.1.0 in
/usr/local/lib/python3.12/dist-packages (from tensorflow) (3.2.0)
Requirement already satisfied: typing-extensions>=3.6.6 in
/usr/local/lib/python3.12/dist-packages (from tensorflow) (4.15.0)
Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.12/dist-
packages (from tensorflow) (2.0.1)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in
/usr/local/lib/python3.12/dist-packages (from tensorflow) (1.76.0)
Requirement already satisfied: tensorboard~=2.19.0 in
/usr/local/lib/python3.12/dist-packages (from tensorflow) (2.19.0)
Requirement already satisfied: keras>=3.5.0 in /usr/local/lib/python3.12/dist-
packages (from tensorflow) (3.10.0)
Requirement already satisfied: numpy<2.2.0,>=1.26.0 in
/usr/local/lib/python3.12/dist-packages (from tensorflow) (2.0.2)
Requirement already satisfied: h5py>=3.11.0 in /usr/local/lib/python3.12/dist-
packages (from tensorflow) (3.15.1)
Requirement already satisfied: ml-dtypes<1.0.0,>=0.5.1 in
/usr/local/lib/python3.12/dist-packages (from tensorflow) (0.5.4)
```

```
Requirement already satisfied: wheel<1.0,>=0.23.0 in
/usr/local/lib/python3.12/dist-packages (from astunparse>=1.6.0->tensorflow)
(0.45.1)
Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-packages
(from keras>=3.5.0->tensorflow) (13.9.4)
Requirement already satisfied: namex in /usr/local/lib/python3.12/dist-packages
(from keras>=3.5.0->tensorflow) (0.1.0)
Requirement already satisfied: optree in /usr/local/lib/python3.12/dist-packages
(from keras>=3.5.0->tensorflow) (0.18.0)
Requirement already satisfied: charset_normalizer<4,>=2 in
/usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow)
(3.4.4)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-
packages (from requests<3,>=2.21.0->tensorflow) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow)
(2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in
/usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow)
(2025.11.12)
Requirement already satisfied: markdown>=2.6.8 in
/usr/local/lib/python3.12/dist-packages (from tensorboard~=2.19.0->tensorflow)
(3.10)
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in
/usr/local/lib/python3.12/dist-packages (from tensorboard~=2.19.0->tensorflow)
(0.7.2)
Requirement already satisfied: werkzeug>=1.0.1 in
/usr/local/lib/python3.12/dist-packages (from tensorboard~=2.19.0->tensorflow)
(3.1.3)
Requirement already satisfied: MarkupSafe>=2.1.1 in
/usr/local/lib/python3.12/dist-packages (from
werkzeug>=1.0.1->tensorboard~=2.19.0->tensorflow) (3.0.3)
Requirement already satisfied: markdown-it-py>=2.2.0 in
/usr/local/lib/python3.12/dist-packages (from rich->keras>=3.5.0->tensorflow)
(4.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in
/usr/local/lib/python3.12/dist-packages (from rich->keras>=3.5.0->tensorflow)
(2.19.2)
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-
packages (from markdown-it-py>=2.2.0->rich->keras>=3.5.0->tensorflow) (0.1.2)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-
packages (1.6.1)
Requirement already satisfied: numpy>=1.19.5 in /usr/local/lib/python3.12/dist-
packages (from scikit-learn) (2.0.2)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-
packages (from scikit-learn) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-
packages (from scikit-learn) (1.5.2)
```

```
Requirement already satisfied: threadpoolctl>=3.1.0 in
/usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
```

[ ]:
```python
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.datasets import cifar10
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras import layers, models


# 1. Load Data
(x_train_full, y_train_full), (x_test_full, y_test_full) = cifar10.load_data()

# Combine to split manually (70% Train, 30% Test)
X_all = np.concatenate((x_train_full, x_test_full))
y_all = np.concatenate((y_train_full, y_test_full))

# 2. Convert to Grayscale and cast to float32 early for consistency
X_gray_raw = tf.image.rgb_to_grayscale(X_all).numpy()
X_gray_float32 = X_gray_raw.astype(np.float32)

# 3. Flatten the images (N, 32, 32, 1) -> (N, 1024)
n_samples = X_gray_float32.shape[0]
X_flat_float32 = X_gray_float32.reshape(n_samples, -1)

# 4. Split 70% Training, 30% Validation/Test
X_train, X_test, y_train, y_test = train_test_split(
    X_flat_float32, y_all, test_size=0.30, random_state=42, stratify=y_all
)

# 5. Normalize (Mean and Variance normalization) and ensure float32 output
scaler = StandardScaler()
X_train_norm = scaler.fit_transform(X_train).astype(np.float32)
X_test_norm = scaler.transform(X_test).astype(np.float32)

y_train = y_train.flatten()
y_test = y_test.flatten()

print(f"Training Shape: {X_train_norm.shape}")
print(f"Testing Shape: {X_test_norm.shape}")
```

```
Training Shape: (42000, 1024)
Testing Shape: (18000, 1024)
```

2 Task 1

[ ]:
```
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_curve, auc
from sklearn.preprocessing import label_binarize
from sklearn.multiclass import OneVsRestClassifier
import matplotlib.pyplot as plt

# 1. Determine K using the 'full' SVD solver with n_components=0.95
pca_full_pre_run = PCA(n_components=0.95, svd_solver='full', random_state=42)
pca_full_pre_run.fit(X_train_norm)
K_fixed = pca_full_pre_run.n_components_
print(f"Components (K) required for 95% energy (determined by 'full' SVD):␣
 ↪{K_fixed}")
print("---")

def perform_pca_task(solver_type, K, X_train, X_test, y_train, y_test):
    print(f"\n--- Running {solver_type} PCA ---")

    # 1. PCA
    pca = PCA(n_components=K, svd_solver=solver_type, random_state=42)
    X_train_pca = pca.fit_transform(X_train)
    X_test_pca = pca.transform(X_test)

    K_actual = pca.n_components_
    print(f"Components (K) used: {K_actual}")

    # 2. Logistic Regression
    # Using multinomial logistic regression
    clf = LogisticRegression(solver='lbfgs', max_iter=1000)
    clf.fit(X_train_pca, y_train)

    # Predict probabilities for ROC
    y_score = clf.predict_proba(X_test_pca)

    return pca, K_actual, y_score

# Run Standard PCA with the fixed K
pca_std, K_std, y_score_std = perform_pca_task('full', K_fixed, X_train_norm,␣
 ↪X_test_norm, y_train, y_test)

# Run Randomized PCA with the fixed K
pca_rnd, K_rnd, y_score_rnd = perform_pca_task('randomized', K_fixed,␣
 ↪X_train_norm, X_test_norm, y_train, y_test)

# --- Plotting ROC Curves ---
```

```
def plot_multiclass_roc(y_test, y_score, title):
    # Binarize the output for ROC calculation (One-vs-Rest)
    y_test_bin = label_binarize(y_test, classes=range(10))
    n_classes = 10

    fpr = dict()
    tpr = dict()
    roc_auc = dict()

    # Calculate ROC for each class
    for i in range(n_classes):
        fpr[i], tpr[i], _ = roc_curve(y_test_bin[:, i], y_score[:, i])
        roc_auc[i] = auc(fpr[i], tpr[i])

    # Compute micro-average ROC curve
    fpr["micro"], tpr["micro"], _ = roc_curve(y_test_bin.ravel(), y_score.
 ↪ravel())
    roc_auc["micro"] = auc(fpr["micro"], tpr["micro"])

    plt.figure(figsize=(8, 6))
    plt.plot(fpr["micro"], tpr["micro"],
             label='micro-average ROC curve (area = {0:0.2f})'.
 ↪format(roc_auc["micro"]),
             color='deeppink', linestyle=':', linewidth=4)

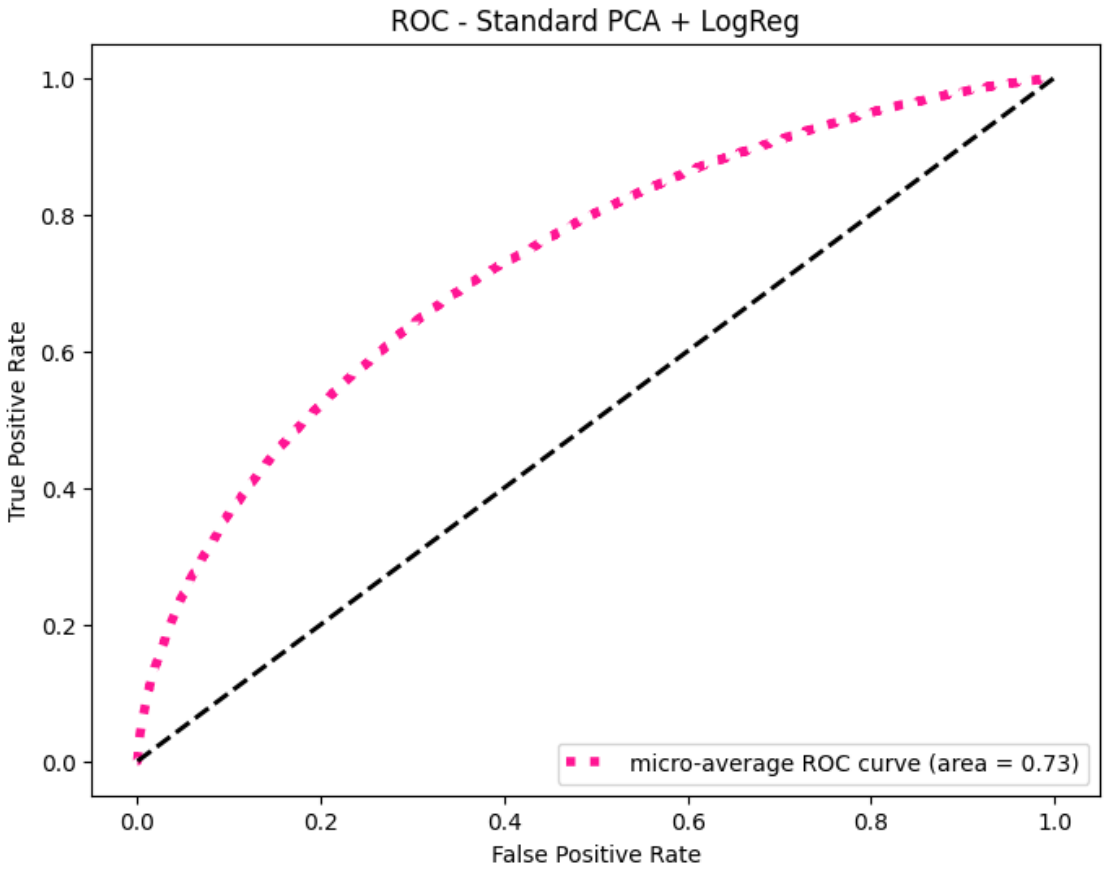
    plt.plot([0, 1], [0, 1], 'k--', lw=2)
    plt.xlabel('False Positive Rate')
    plt.ylabel('True Positive Rate')
    plt.title(title)
    plt.legend(loc="lower right")
    plt.show()

plot_multiclass_roc(y_test, y_score_std, "ROC - Standard PCA + LogReg")
```

```
Components (K) required for 95% energy (determined by 'full' SVD): 163
---

--- Running full PCA ---
Components (K) used: 163

--- Running randomized PCA ---
Components (K) used: 163
```



3 Task 2

[ ]:
```
from tensorflow.keras import layers, models, constraints, backend as K

class DenseTied(layers.Layer):
    def __init__(self, units, tied_to, **kwargs):
        super(DenseTied, self).__init__(**kwargs)
        self.units = units
        self.tied_to = tied_to

    def build(self, input_shape):
        super(DenseTied, self).build(input_shape)

    def call(self, inputs):
        weights = K.transpose(self.tied_to.kernel)
        return K.dot(inputs, weights)

# --- Build the Autoencoder ---
input_dim = 1024 # 32x32
```

```
encoding_dim = K_std # Use K from Task 1

inputs = layers.Input(shape=(input_dim,))

# Encoder: Linear activation + Unit Norm constraint
encoder_layer = layers.Dense(
    encoding_dim,
    activation='linear',
    use_bias=False,
    kernel_constraint=constraints.UnitNorm(axis=0),
    name="encoder"
)
encoded = encoder_layer(inputs)

# Decoder: Tied weights (Transpose of encoder)
decoded = DenseTied(units=input_dim, tied_to=encoder_layer)(encoded)

autoencoder = models.Model(inputs, decoded)

# Compile
autoencoder.compile(optimizer='adam', loss='mean_squared_error')

# Train
history = autoencoder.fit(
    X_train_norm, X_train_norm,
    epochs=10,
    batch_size=256,
    shuffle=True,
    validation_data=(X_test_norm, X_test_norm),
    verbose=1
)
```

```
Epoch 1/10
165/165            3s 12ms/step -
loss: 0.3403 - val_loss: 0.1088
Epoch 2/10
165/165            1s 4ms/step -
loss: 0.0965 - val_loss: 0.0760
Epoch 3/10
165/165            1s 4ms/step -
loss: 0.0711 - val_loss: 0.0636
Epoch 4/10
165/165            1s 4ms/step -
loss: 0.0610 - val_loss: 0.0579
Epoch 5/10
165/165            1s 4ms/step -
loss: 0.0563 - val_loss: 0.0552
```

```
Epoch 6/10
165/165             1s 4ms/step -
loss: 0.0540 - val_loss: 0.0540
Epoch 7/10
165/165             1s 4ms/step -
loss: 0.0530 - val_loss: 0.0534
Epoch 8/10
165/165             1s 5ms/step -
loss: 0.0526 - val_loss: 0.0531
Epoch 9/10
165/165             1s 5ms/step -
loss: 0.0523 - val_loss: 0.0529
Epoch 10/10
165/165             1s 4ms/step -
loss: 0.0522 - val_loss: 0.0528
```

4 compare eigen vectors from task 1 with autoencoders

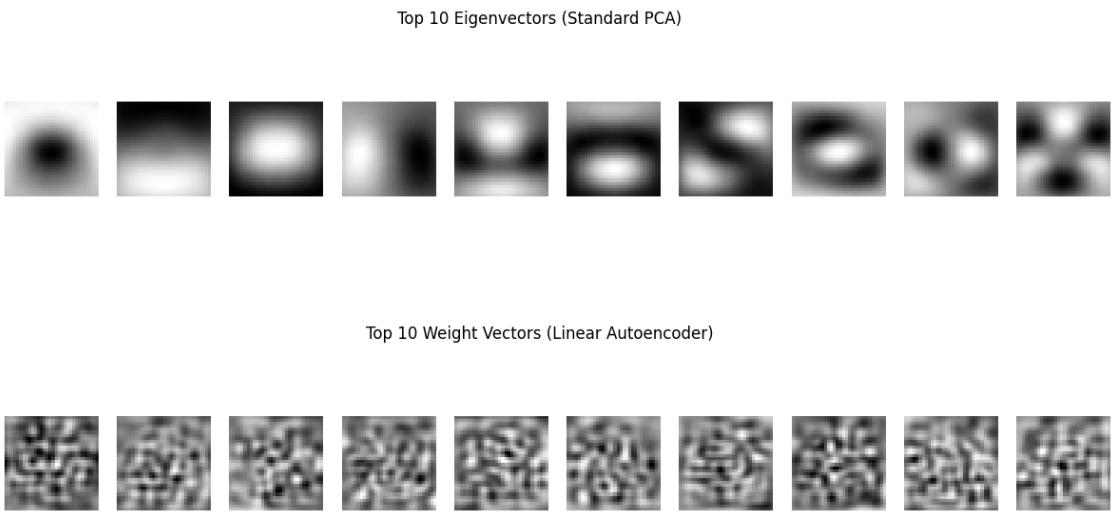
[ ]:
```python
# --- Visualization of Components ---

def plot_components(components, title, n_images=10):
    plt.figure(figsize=(15, 3))
    for i in range(n_images):
        ax = plt.subplot(1, n_images, i + 1)
        img = components[i].reshape(32, 32)
        plt.imshow(img, cmap='gray')
        plt.axis('off')
    plt.suptitle(title)
    plt.show()

# 1. Extract PCA Eigenvectors (Eigenfaces)
# PCA components_ shape is (n_components, n_features)
pca_components = pca_std.components_

# 2. Extract Autoencoder Weights
# AE Weights shape is (n_features, n_hidden). We need to transpose to match PCA␣
 ↪shape.
ae_weights = encoder_layer.get_weights()[0].T

# Plot
plot_components(pca_components, "Top 10 Eigenvectors (Standard PCA)")
plot_components(ae_weights, "Top 10 Weight Vectors (Linear Autoencoder)")
```



Observation: There is a visual difference between the two rows:

Row 1 (PCA): The images are smooth, structured, and geometric. They resemble gradients, blobs, and wave like patterns.

Row 2 (Autoencoder): The images appear "noisy," granular, or like "TV static," lacking distinct global shapes.

Analysis & Reasoning

1. PCA: Ordered Variance and Orthogonality

The images in the top row are Eigenfaces. PCA works sequentially to maximize variance.The first image (leftmost) captures the most dominant feature (likely the background illumination or the average circular shape of objects centered in CIFAR images).The subsequent images capture the next most important directions of variance, orthogonal to the previous ones.Because PCA concentrates the "energy" (information) into the earliest components, these vectors represent low frequency structures (smooth changes in lighting and shape), resulting in the blurry, smooth appearance.

2. Linear Autoencoder: Subspace Rotation (The "Why" for the Noise)We might expect the Autoencoder weights to look exactly like the PCA eigenvectors because a Linear Autoencoder with tied weights optimizes a similar objective to PCA (minimizing reconstruction error). However, they look noisy because there is no ordering constraint. Unlike PCA, the Autoencoder does not force the first node to be more important than the second. It optimizes all K nodes simultaneously.

Rotational Invariance: While the Autoencoder learns the same subspace as PCA (it covers the same mathematical ground), it finds a different basis to describe that space.

Distributed Representation: The Autoencoder "mixes" the PCA components together. Instead of having one lighting vector and one edge vector, the Autoencoder spreads this information across all the weight vectors. This mixing results in the noisy or high frequency appearance, as every weight vector is doing a little bit of work to reconstruct the image details.

Conclusion:

While both the PCA eigenvectors and the Linear Autoencoder weights span the same principal subspace, their visual representations differ significantly. PCA enforces an orthogonality and variance maximization constraint, resulting in smooth, and interpretable eigen-images.In contrast, the Linear Autoencoder lacks the constraint to align its weights along the axes of maximum variance. Consequently, it learns an arbitrary rotation of the PCA basis, distributing the image features (variance) continuously across all nodes, leading to weight vectors that appear visually mixed or granular."

5 Task 3

[ ]:
```
K = 64  # <<< IMPORTANT: set to the SAME K used in earlier tasks
batch_size = 256
epochs = 50
random_state = 42

np.random.seed(random_state)
tf.random.set_seed(random_state)
```

[ ]:
```
input_shape = X_train.shape[1:]          # (32, 32, 1)
flat_dim = np.prod(input_shape).item()

# Single hidden layer dense autoencoder (baseline)
#    - One hidden layer with K nodes, sigmoid activation
#    - Output layer linear
def build_single_hidden_autoencoder(flat_dim, K):
    inp = layers.Input(shape=(flat_dim,), name="input_flat")
    hidden = layers.Dense(K, activation="sigmoid", name="hidden_sigmoid")(inp)
    out = layers.Dense(flat_dim, activation="linear",␣
 ↪name="output_linear")(hidden)

    model = models.Model(inp, out, name="single_hidden_autoencoder")
    model.compile(optimizer=tf.keras.optimizers.Adam(),
                  loss="mse")  # MSE = reconstruction error
    return model
```

[ ]:
```
# 5. Three-hidden-layer dense autoencoder
#    - Total K hidden nodes split across 3 layers
#      e.g., if K=64 -> [21, 21, 22]
#    - All hidden: sigmoid
#    - Output: linear
def build_three_hidden_autoencoder(flat_dim, K):
    # Distribute K nodes approximately equally across 3 layers
    base = K // 3
    sizes = [base, base, K - 2 * base]  # sum(sizes) = K
```

```
    inp = layers.Input(shape=(flat_dim,), name="input_flat")
    h1 = layers.Dense(sizes[0], activation="sigmoid",␣
 ↪name="hidden1_sigmoid")(inp)
    h2 = layers.Dense(sizes[1], activation="sigmoid",␣
 ↪name="hidden2_sigmoid")(h1)
    h3 = layers.Dense(sizes[2], activation="sigmoid",␣
 ↪name="hidden3_sigmoid")(h2)
    out = layers.Dense(flat_dim, activation="linear", name="output_linear")(h3)

    model = models.Model(inp, out, name="three_hidden_autoencoder")
    model.compile(optimizer=tf.keras.optimizers.Adam(),
                  loss="mse")
    return model
```

[ ]:
```
# 6. Deep convolutional autoencoder
#    - Latent space dimension  K
#    - Conv encoder, Dense(K) bottleneck
#    - ConvTranspose decoder
#    - Output: linear
def build_conv_autoencoder(input_shape, K):
    inp = layers.Input(shape=input_shape, name="input_img", dtype=tf.float32)

    # ----- Encoder -----
    # 32x32x1 -> 16x16x32
    x = layers.Conv2D(32, (3, 3), strides=2, padding="same",
                      activation="relu", name="enc_conv1")(inp)
    # 16x16x32 -> 8x8x64
    x = layers.Conv2D(64, (3, 3), strides=2, padding="same",
                      activation="relu", name="enc_conv2")(x)

    # Flatten and bottleneck
    x_flat = layers.Flatten(name="flatten")(x)
    latent = layers.Dense(K, activation="relu", name="latent")(x_flat)

    # ----- Decoder -----
    # Map back up to 8x8x64
    x_dec = layers.Dense(8 * 8 * 64, activation="relu",␣
 ↪name="dec_dense")(latent)
    x_dec = layers.Reshape((8, 8, 64), name="reshape_dec")(x_dec)

    # 8x8x64 -> 16x16x64
    x_dec = layers.Conv2DTranspose(64, (3, 3), strides=2, padding="same",
                                   activation="relu", name="dec_deconv1")(x_dec)
    # 16x16x64 -> 32x32x32
    x_dec = layers.Conv2DTranspose(32, (3, 3), strides=2, padding="same",
                                   activation="relu", name="dec_deconv2")(x_dec)
```

```
    # Final reconstruction: 32x32x1
    # Linear activation as requested for decoder final layer
    out = layers.Conv2D(1, (3, 3), padding="same",
                        activation="linear", name="output_linear")(x_dec)

    model = models.Model(inp, out, name="conv_autoencoder")
    model.compile(optimizer=tf.keras.optimizers.Adam(),
                  loss="mse")  # reconstruction error
    return model
```

[ ]:
```
# 7. Prepare data for dense models (flattened)
x_train_flat = X_train_norm.reshape((X_train_norm.shape[0], flat_dim))
x_test_flat = X_test_norm.reshape((X_test_norm.shape[0], flat_dim))
```

[ ]:
```
# 8. Build models
single_ae = build_single_hidden_autoencoder(flat_dim, K)
three_ae = build_three_hidden_autoencoder(flat_dim, K)
# The input_shape for the convolutional autoencoder should be (height, width,␣
 ↪channels),
# which is (32, 32, 1) for grayscale CIFAR10 images.
# The original 'input_shape' variable holds (1024,), which is for flattened␣
 ↪data.
conv_ae = build_conv_autoencoder((32, 32, 1), K)

single_ae.summary()
three_ae.summary()
conv_ae.summary()
```

Model: "single_hidden_autoencoder"

Layer (type)	Output Shape	Param #
input_flat (InputLayer)	(None, 1024)	0
hidden_sigmoid (Dense)	(None, 64)	65,600
output_linear (Dense)	(None, 1024)	66,560

Total params: 132,160 (516.25 KB)

Trainable params: 132,160 (516.25 KB)

Non-trainable params: 0 (0.00 B)

Model: "three_hidden_autoencoder"

Layer (type)	Output Shape	Param #
input_flat (InputLayer)	(None, 1024)	0
hidden1_sigmoid (Dense)	(None, 21)	21,525
hidden2_sigmoid (Dense)	(None, 21)	462
hidden3_sigmoid (Dense)	(None, 22)	484
output_linear (Dense)	(None, 1024)	23,552

Total params: 46,023 (179.78 KB)

Trainable params: 46,023 (179.78 KB)

Non-trainable params: 0 (0.00 B)

Model: "conv_autoencoder"

Layer (type)	Output Shape	Param #
input_img (InputLayer)	(None, 32, 32, 1)	0
enc_conv1 (Conv2D)	(None, 16, 16, 32)	320
enc_conv2 (Conv2D)	(None, 8, 8, 64)	18,496
flatten (Flatten)	(None, 4096)	0
latent (Dense)	(None, 64)	262,208
dec_dense (Dense)	(None, 4096)	266,240
reshape_dec (Reshape)	(None, 8, 8, 64)	0
dec_deconv1 (Conv2DTranspose)	(None, 16, 16, 64)	36,928

```
 dec_deconv2 (Conv2DTranspose)     (None, 32, 32, 32)              18,464

 output_linear (Conv2D)            (None, 32, 32, 1)                 289


Total params: 602,945 (2.30 MB)

Trainable params: 602,945 (2.30 MB)

Non-trainable params: 0 (0.00 B)
```

[ ]:
```
# 9. Train models (with early stopping)
early_stop = tf.keras.callbacks.EarlyStopping(
    monitor="val_loss",
    patience=5,
    restore_best_weights=True
)

print("\nTraining single-hidden-layer dense autoencoder...")
history_single = single_ae.fit(
    x_train_flat, x_train_flat,
    validation_data=(x_test_flat, x_test_flat),
    epochs=epochs,
    batch_size=batch_size,
    callbacks=[early_stop],
    verbose=1
)

print("\nTraining three-hidden-layer dense autoencoder...")
history_three = three_ae.fit(
    x_train_flat, x_train_flat,
    validation_data=(x_test_flat, x_test_flat),
    epochs=epochs,
    batch_size=batch_size,
    callbacks=[early_stop],
    verbose=1
)

# Reshape data for convolutional autoencoder, using normalized data and casting␣
 ↪to float32
X_train_conv = X_train_norm.reshape(-1, 32, 32, 1).astype(np.float32)
X_test_conv = X_test_norm.reshape(-1, 32, 32, 1).astype(np.float32)

print("\nTraining convolutional autoencoder...")
history_conv = conv_ae.fit(
```

```
    X_train_conv, X_train_conv,
    validation_data=(X_test_conv, X_test_conv),
    epochs=epochs,
    batch_size=batch_size,
    callbacks=[early_stop],
    verbose=1
)
```

```
Training single-hidden-layer dense autoencoder…
Epoch 1/50
165/165             3s 13ms/step -
loss: 0.7115 - val_loss: 0.4011
Epoch 2/50
165/165             1s 4ms/step -
loss: 0.3671 - val_loss: 0.3079
Epoch 3/50
165/165             1s 4ms/step -
loss: 0.2907 - val_loss: 0.2638
Epoch 4/50
165/165             1s 4ms/step -
loss: 0.2510 - val_loss: 0.2367
Epoch 5/50
165/165             1s 4ms/step -
loss: 0.2253 - val_loss: 0.2194
Epoch 6/50
165/165             1s 4ms/step -
loss: 0.2073 - val_loss: 0.2031
Epoch 7/50
165/165             1s 4ms/step -
loss: 0.1936 - val_loss: 0.1907
Epoch 8/50
165/165             1s 4ms/step -
loss: 0.1825 - val_loss: 0.1804
Epoch 9/50
165/165             1s 4ms/step -
loss: 0.1737 - val_loss: 0.1728
Epoch 10/50
165/165             1s 4ms/step -
loss: 0.1666 - val_loss: 0.1665
Epoch 11/50
165/165             1s 4ms/step -
loss: 0.1610 - val_loss: 0.1612
Epoch 12/50
165/165             1s 4ms/step -
loss: 0.1563 - val_loss: 0.1569
Epoch 13/50
```

```
165/165            1s 5ms/step -
loss: 0.1525 - val_loss: 0.1532
Epoch 14/50
165/165            1s 6ms/step -
loss: 0.1493 - val_loss: 0.1502
Epoch 15/50
165/165            1s 5ms/step -
loss: 0.1466 - val_loss: 0.1479
Epoch 16/50
165/165            1s 4ms/step -
loss: 0.1444 - val_loss: 0.1462
Epoch 17/50
165/165            1s 4ms/step -
loss: 0.1425 - val_loss: 0.1446
Epoch 18/50
165/165            1s 4ms/step -
loss: 0.1409 - val_loss: 0.1432
Epoch 19/50
165/165            1s 4ms/step -
loss: 0.1395 - val_loss: 0.1418
Epoch 20/50
165/165            1s 4ms/step -
loss: 0.1383 - val_loss: 0.1406
Epoch 21/50
165/165            1s 4ms/step -
loss: 0.1372 - val_loss: 0.1394
Epoch 22/50
165/165            1s 4ms/step -
loss: 0.1363 - val_loss: 0.1383
Epoch 23/50
165/165            1s 4ms/step -
loss: 0.1354 - val_loss: 0.1374
Epoch 24/50
165/165            1s 4ms/step -
loss: 0.1346 - val_loss: 0.1366
Epoch 25/50
165/165            1s 4ms/step -
loss: 0.1338 - val_loss: 0.1359
Epoch 26/50
165/165            1s 4ms/step -
loss: 0.1331 - val_loss: 0.1353
Epoch 27/50
165/165            1s 4ms/step -
loss: 0.1325 - val_loss: 0.1348
Epoch 28/50
165/165            1s 5ms/step -
loss: 0.1320 - val_loss: 0.1343
Epoch 29/50
```

```
165/165            1s 6ms/step -
loss: 0.1315 - val_loss: 0.1339
Epoch 30/50
165/165            1s 6ms/step -
loss: 0.1309 - val_loss: 0.1334
Epoch 31/50
165/165            1s 4ms/step -
loss: 0.1304 - val_loss: 0.1327
Epoch 32/50
165/165            1s 4ms/step -
loss: 0.1300 - val_loss: 0.1324
Epoch 33/50
165/165            1s 4ms/step -
loss: 0.1296 - val_loss: 0.1320
Epoch 34/50
165/165            1s 4ms/step -
loss: 0.1293 - val_loss: 0.1317
Epoch 35/50
165/165            1s 8ms/step -
loss: 0.1290 - val_loss: 0.1314
Epoch 36/50
165/165            2s 4ms/step -
loss: 0.1287 - val_loss: 0.1311
Epoch 37/50
165/165            1s 4ms/step -
loss: 0.1284 - val_loss: 0.1308
Epoch 38/50
165/165            1s 4ms/step -
loss: 0.1281 - val_loss: 0.1305
Epoch 39/50
165/165            1s 4ms/step -
loss: 0.1279 - val_loss: 0.1303
Epoch 40/50
165/165            1s 4ms/step -
loss: 0.1276 - val_loss: 0.1301
Epoch 41/50
165/165            1s 5ms/step -
loss: 0.1274 - val_loss: 0.1299
Epoch 42/50
165/165            1s 6ms/step -
loss: 0.1272 - val_loss: 0.1296
Epoch 43/50
165/165            1s 7ms/step -
loss: 0.1270 - val_loss: 0.1296
Epoch 44/50
165/165            1s 7ms/step -
loss: 0.1267 - val_loss: 0.1292
Epoch 45/50
```

```
165/165             1s 6ms/step -
loss: 0.1264 - val_loss: 0.1291
Epoch 46/50
165/165             1s 4ms/step -
loss: 0.1262 - val_loss: 0.1288
Epoch 47/50
165/165             1s 4ms/step -
loss: 0.1262 - val_loss: 0.1289
Epoch 48/50
165/165             1s 4ms/step -
loss: 0.1260 - val_loss: 0.1285
Epoch 49/50
165/165             1s 4ms/step -
loss: 0.1259 - val_loss: 0.1284
Epoch 50/50
165/165             1s 4ms/step -
loss: 0.1256 - val_loss: 0.1282

Training three-hidden-layer dense autoencoder…
Epoch 1/50
165/165             4s 17ms/step -
loss: 0.9572 - val_loss: 0.7586
Epoch 2/50
165/165             1s 4ms/step -
loss: 0.7351 - val_loss: 0.6531
Epoch 3/50
165/165             1s 5ms/step -
loss: 0.6319 - val_loss: 0.6140
Epoch 4/50
165/165             1s 4ms/step -
loss: 0.5977 - val_loss: 0.5559
Epoch 5/50
165/165             1s 4ms/step -
loss: 0.5449 - val_loss: 0.5371
Epoch 6/50
165/165             1s 4ms/step -
loss: 0.5315 - val_loss: 0.5309
Epoch 7/50
165/165             1s 4ms/step -
loss: 0.5264 - val_loss: 0.5295
Epoch 8/50
165/165             1s 4ms/step -
loss: 0.5194 - val_loss: 0.5069
Epoch 9/50
165/165             1s 4ms/step -
loss: 0.5001 - val_loss: 0.4936
Epoch 10/50
165/165             1s 4ms/step -
```

```
loss: 0.4895 - val_loss: 0.4859
Epoch 11/50
165/165             1s 4ms/step -
loss: 0.4796 - val_loss: 0.4714
Epoch 12/50
165/165             1s 4ms/step -
loss: 0.4631 - val_loss: 0.4622
Epoch 13/50
165/165             1s 4ms/step -
loss: 0.4560 - val_loss: 0.4575
Epoch 14/50
165/165             1s 4ms/step -
loss: 0.4526 - val_loss: 0.4560
Epoch 15/50
165/165             1s 6ms/step -
loss: 0.4511 - val_loss: 0.4532
Epoch 16/50
165/165             1s 6ms/step -
loss: 0.4495 - val_loss: 0.4518
Epoch 17/50
165/165             1s 5ms/step -
loss: 0.4483 - val_loss: 0.4506
Epoch 18/50
165/165             1s 4ms/step -
loss: 0.4473 - val_loss: 0.4498
Epoch 19/50
165/165             1s 5ms/step -
loss: 0.4465 - val_loss: 0.4493
Epoch 20/50
165/165             1s 4ms/step -
loss: 0.4459 - val_loss: 0.4494
Epoch 21/50
165/165             1s 4ms/step -
loss: 0.4454 - val_loss: 0.4488
Epoch 22/50
165/165             1s 4ms/step -
loss: 0.4450 - val_loss: 0.4489
Epoch 23/50
165/165             1s 4ms/step -
loss: 0.4449 - val_loss: 0.4484
Epoch 24/50
165/165             1s 4ms/step -
loss: 0.4450 - val_loss: 0.4487
Epoch 25/50
165/165             1s 4ms/step -
loss: 0.4437 - val_loss: 0.4481
Epoch 26/50
165/165             1s 4ms/step -
```

```
loss: 0.4431 - val_loss: 0.4474
Epoch 27/50
165/165             1s 4ms/step -
loss: 0.4421 - val_loss: 0.4461
Epoch 28/50
165/165             1s 4ms/step -
loss: 0.4403 - val_loss: 0.4422
Epoch 29/50
165/165             1s 5ms/step -
loss: 0.4344 - val_loss: 0.4322
Epoch 30/50
165/165             1s 6ms/step -
loss: 0.4265 - val_loss: 0.4273
Epoch 31/50
165/165             1s 5ms/step -
loss: 0.4217 - val_loss: 0.4228
Epoch 32/50
165/165             1s 5ms/step -
loss: 0.4150 - val_loss: 0.4116
Epoch 33/50
165/165             1s 4ms/step -
loss: 0.4029 - val_loss: 0.4013
Epoch 34/50
165/165             1s 4ms/step -
loss: 0.3961 - val_loss: 0.3973
Epoch 35/50
165/165             1s 5ms/step -
loss: 0.3933 - val_loss: 0.3956
Epoch 36/50
165/165             1s 4ms/step -
loss: 0.3921 - val_loss: 0.3947
Epoch 37/50
165/165             1s 4ms/step -
loss: 0.3910 - val_loss: 0.3938
Epoch 38/50
165/165             1s 4ms/step -
loss: 0.3897 - val_loss: 0.3927
Epoch 39/50
165/165             1s 4ms/step -
loss: 0.3892 - val_loss: 0.3920
Epoch 40/50
165/165             1s 5ms/step -
loss: 0.3888 - val_loss: 0.3914
Epoch 41/50
165/165             1s 4ms/step -
loss: 0.3884 - val_loss: 0.3910
Epoch 42/50
165/165             1s 4ms/step -
```

```
loss: 0.3879 - val_loss: 0.3911
Epoch 43/50
165/165             1s 5ms/step -
loss: 0.3870 - val_loss: 0.3908
Epoch 44/50
165/165             1s 6ms/step -
loss: 0.3869 - val_loss: 0.3902
Epoch 45/50
165/165             1s 6ms/step -
loss: 0.3864 - val_loss: 0.3891
Epoch 46/50
165/165             1s 4ms/step -
loss: 0.3851 - val_loss: 0.3877
Epoch 47/50
165/165             1s 4ms/step -
loss: 0.3823 - val_loss: 0.3836
Epoch 48/50
165/165             1s 4ms/step -
loss: 0.3773 - val_loss: 0.3822
Epoch 49/50
165/165             1s 4ms/step -
loss: 0.3726 - val_loss: 0.3777
Epoch 50/50
165/165             1s 4ms/step -
loss: 0.3695 - val_loss: 0.3742

Training convolutional autoencoder…
Epoch 1/50
165/165             10s 36ms/step -
loss: 0.6341 - val_loss: 0.2877
Epoch 2/50
165/165             4s 16ms/step -
loss: 0.2257 - val_loss: 0.2049
Epoch 3/50
165/165             2s 15ms/step -
loss: 0.1785 - val_loss: 0.1695
Epoch 4/50
165/165             3s 16ms/step -
loss: 0.1617 - val_loss: 0.1640
Epoch 5/50
165/165             2s 15ms/step -
loss: 0.1578 - val_loss: 0.1591
Epoch 6/50
165/165             3s 15ms/step -
loss: 0.1534 - val_loss: 0.1551
Epoch 7/50
165/165             2s 15ms/step -
loss: 0.1516 - val_loss: 0.1581
```

```
Epoch 8/50
165/165             3s 15ms/step -
loss: 0.1503 - val_loss: 0.1509
Epoch 9/50
165/165             3s 16ms/step -
loss: 0.1487 - val_loss: 0.1505
Epoch 10/50
165/165             3s 15ms/step -
loss: 0.1471 - val_loss: 0.1500
Epoch 11/50
165/165             2s 15ms/step -
loss: 0.1465 - val_loss: 0.1504
Epoch 12/50
165/165             3s 15ms/step -
loss: 0.1456 - val_loss: 0.1496
Epoch 13/50
165/165             3s 16ms/step -
loss: 0.1448 - val_loss: 0.1473
Epoch 14/50
165/165             3s 16ms/step -
loss: 0.1447 - val_loss: 0.1469
Epoch 15/50
165/165             3s 15ms/step -
loss: 0.1435 - val_loss: 0.1528
Epoch 16/50
165/165             2s 15ms/step -
loss: 0.1439 - val_loss: 0.1482
Epoch 17/50
165/165             3s 15ms/step -
loss: 0.1434 - val_loss: 0.1503
Epoch 18/50
165/165             3s 16ms/step -
loss: 0.1433 - val_loss: 0.1518
Epoch 19/50
165/165             3s 16ms/step -
loss: 0.1425 - val_loss: 0.1445
Epoch 20/50
165/165             3s 15ms/step -
loss: 0.1413 - val_loss: 0.1447
Epoch 21/50
165/165             3s 15ms/step -
loss: 0.1410 - val_loss: 0.1450
Epoch 22/50
165/165             3s 15ms/step -
loss: 0.1417 - val_loss: 0.1438
Epoch 23/50
165/165             3s 16ms/step -
loss: 0.1408 - val_loss: 0.1499
```

```
Epoch 24/50
165/165             3s 15ms/step -
loss: 0.1411 - val_loss: 0.1437
Epoch 25/50
165/165             3s 15ms/step -
loss: 0.1411 - val_loss: 0.1492
Epoch 26/50
165/165             3s 15ms/step -
loss: 0.1425 - val_loss: 0.1426
Epoch 27/50
165/165             3s 15ms/step -
loss: 0.1394 - val_loss: 0.1423
Epoch 28/50
165/165             3s 16ms/step -
loss: 0.1394 - val_loss: 0.1559
Epoch 29/50
165/165             3s 15ms/step -
loss: 0.1437 - val_loss: 0.1456
Epoch 30/50
165/165             3s 15ms/step -
loss: 0.1395 - val_loss: 0.1503
Epoch 31/50
165/165             3s 15ms/step -
loss: 0.1387 - val_loss: 0.1402
Epoch 32/50
165/165             3s 16ms/step -
loss: 0.1369 - val_loss: 0.1400
Epoch 33/50
165/165             3s 16ms/step -
loss: 0.1365 - val_loss: 0.1399
Epoch 34/50
165/165             3s 15ms/step -
loss: 0.1367 - val_loss: 0.1418
Epoch 35/50
165/165             3s 15ms/step -
loss: 0.1365 - val_loss: 0.1401
Epoch 36/50
165/165             3s 15ms/step -
loss: 0.1363 - val_loss: 0.1393
Epoch 37/50
165/165             3s 16ms/step -
loss: 0.1360 - val_loss: 0.1482
Epoch 38/50
165/165             3s 16ms/step -
loss: 0.1370 - val_loss: 0.1398
Epoch 39/50
165/165             3s 15ms/step -
loss: 0.1368 - val_loss: 0.1440
```

```
Epoch 40/50
165/165             3s 15ms/step -
loss: 0.1365 - val_loss: 0.1420
Epoch 41/50
165/165             2s 15ms/step -
loss: 0.1368 - val_loss: 0.1401
```

[ ]:
```
# 10. Compute reconstruction error (MSE) on test set
mse_single = single_ae.evaluate(x_test_flat, x_test_flat, verbose=0)
mse_three = three_ae.evaluate(x_test_flat, x_test_flat, verbose=0)
mse_conv = conv_ae.evaluate(X_test_conv, X_test_conv, verbose=0)

print("\n================ Reconstruction Error (MSE) ================")
print(f"Single hidden layer (K nodes, sigmoid + linear output):   {mse_single:.
 ↪6f}")
print(f"Three hidden layers (total K nodes, sigmoid + linear):    {mse_three:.
 ↪6f}")
print(f"Deep convolutional autoencoder (latent dim \u2248 K):          ␣
 ↪{mse_conv:.6f}")
print("============================================================")
```

```
================ Reconstruction Error (MSE) ================
Single hidden layer (K nodes, sigmoid + linear output):   0.128155
Three hidden layers (total K nodes, sigmoid + linear):    0.374188
Deep convolutional autoencoder (latent dim  K):          0.139345
============================================================
```

6 Task 4

[ ]:
```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as patches
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, losses
from tensorflow.keras.models import Model
from tensorflow.keras.models import Sequential #for MLP classifer
from tensorflow.keras.layers import Dense, Dropout
from tensorflow import keras # Import keras for regularizers
from keras.datasets import mnist
from keras.layers import Input, Dense, Conv2D, MaxPooling2D, UpSampling2D
from keras.models import Model
from keras import backend as K
```

[ ]:
```
#Load the MNIST dataset
(x_train, y_train), (x_test, y_test) = mnist.load_data()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-
datasets/mnist.npz
11490434/11490434              2s
0us/step
```

[ ]:
```
#explore the dataset
print(x_train.shape)
print(y_train.shape)
print(x_test.shape)
print(y_test.shape)
```

```
(60000, 28, 28)
(60000,)
(10000, 28, 28)
(10000,)
```

[ ]:
```
x_train[2]
```

[ ]:
```
array([[  0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
          0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
          0,   0],
       [
```

```
   0,   0],
[  0,   0,   0,  46, 245, 163,   0,   0,   0,   0,   0,   0,   0,
   0,   0,   0,   0,   0,   0, 198, 254,  56,   0,   0,   0,   0,
   0,   0],
[  0,   0,   0, 120, 254, 163,   0,   0,   0,   0,   0,   0,   0,
   0,   0,   0,   0,   0,  23, 231, 254,  29,   0,   0,   0,   0,
   0,   0],
[  0,   0,   0, 159, 254, 120,   0,   0,   0,   0,   0,   0,   0,
   0,   0,   0,   0,   0, 163, 254, 216,  16,   0,   0,   0,   0,
   0,   0],
[  0,   0,   0, 159, 254,  67,   0,   0,   0,   0,   0,   0,   0,
   0,   0,  14,  86, 178, 248, 254,  91,   0,   0,   0,   0,   0,
   0,   0],
[  0,   0,   0, 159, 254,  85,   0,   0,   0,  47,  49, 116, 144,
 150, 241, 243, 234, 179, 241, 252,  40,   0,   0,   0,   0,   0,
   0,   0],
[  0,   0,   0, 150, 253, 237, 207, 207, 207, 253, 254, 250, 240,
 198, 143,  91,  28,   5, 233, 250,   0,   0,   0,   0,   0,   0,
   0,   0],
[  0,   0,   0,   0, 119, 177, 177, 177, 177, 177,  98,  56,   0,
   0,   0,   0,   0, 102, 254, 220,   0,   0,   0,   0,   0,   0,
   0,   0],
[  0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
   0,   0,   0,   0, 169, 254, 137,   0,   0,   0,   0,   0,   0,
   0,   0],
[  0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
   0,   0,   0,   0, 169, 254,  57,   0,   0,   0,   0,   0,   0,
   0,   0],
[  0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
   0,   0,   0,   0, 169, 254,  57,   0,   0,   0,   0,   0,   0,
   0,   0],
[  0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
   0,   0,   0,   0, 169, 255,  94,   0,   0,   0,   0,   0,   0,
   0,   0],
[  0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
   0,   0,   0,   0, 169, 254,  96,   0,   0,   0,   0,   0,   0,
   0,   0],
[  0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
   0,   0,   0,   0, 169, 254, 153,   0,   0,   0,   0,   0,   0,
   0,   0],
[  0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
   0,   0,   0,   0, 169, 255, 153,   0,   0,   0,   0,   0,   0,
   0,   0],
[  0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
   0,   0,   0,  96, 254, 153,   0,   0,   0,   0,   0,   0,   0,
   0,   0],
[  0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
```

```
        0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
        0,   0],
       [ 0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
        0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
        0,   0],
       [ 0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
        0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
        0,   0]], dtype=uint8)
```

6.0.1 Train a deep convolutional autoencoder

[ ]:
```
#reshape data to include channel dimension

x_train_reshape = x_train.reshape((len(x_train), 28, 28, 1))
x_test_reshape = x_test.reshape((len(x_test), 28, 28, 1))
print(f"Shape of x_train after reshaping: {x_train_reshape.shape}")
print(f"Shape of x_test after reshaping: {x_test_reshape.shape}")
```

```
Shape of x_train after reshaping: (60000, 28, 28, 1)
Shape of x_test after reshaping: (10000, 28, 28, 1)
```

[ ]:
```
#define ENCODER

# Define input_img
input_img = Input(shape=(28, 28, 1))

conv1 = Conv2D(32, (3, 3), activation='relu', padding='same')(input_img)
pool1 = MaxPooling2D((2, 2), padding='same')(conv1)
conv2 = Conv2D(32, (3, 3), activation='relu', padding='same')(pool1)
pool2 = MaxPooling2D((2, 2), padding='same')(conv2)
encoded = Conv2D(16, (3, 3), activation='relu', padding='same')(pool2)
```

[ ]:
```
#define DECODER

conv3 = Conv2D(32, (3, 3), activation='relu', padding='same')(encoded)
up1 = UpSampling2D((2, 2))(conv3)
conv4 = Conv2D(32, (3, 3), activation='relu', padding='same')(up1)
up2 = UpSampling2D((2, 2))(conv4)
decoded = Conv2D(1, (3, 3), activation='sigmoid', padding='same')(up2)
```

[ ]:
```
#build auto-encoder model
autoencoder = Model(input_img, decoded)

#Compile the autoencoder model with Adam optimizer and loss function
autoencoder.compile(optimizer=keras.optimizers.Adam(learning_rate=1e-3),␣
 ↪loss='binary_crossentropy', metrics=['accuracy'], run_eagerly=True)
```

[ ]:
```
# Normalize image data to [0, 1] range and convert to float32
x_train_normalized = x_train_reshape.astype('float32') / 255.
x_test_normalized = x_test_reshape.astype('float32') / 255.

print(f"Shape of normalized x_train: {x_train_normalized.shape}, Dtype:␣
 ↪{x_train_normalized.dtype}")
print(f"Shape of normalized x_test: {x_test_normalized.shape}, Dtype:␣
 ↪{x_test_normalized.dtype}")
```

```
Shape of normalized x_train: (60000, 28, 28, 1), Dtype: float32
Shape of normalized x_test: (10000, 28, 28, 1), Dtype: float32
```

[ ]:
```
# Create TensorFlow Dataset objects for training and validation
train_dataset = tf.data.Dataset.from_tensor_slices((x_train_normalized,␣
 ↪x_train_normalized)).batch(128)
test_dataset = tf.data.Dataset.from_tensor_slices((x_test_normalized,␣
 ↪x_test_normalized)).batch(128)

autoencoder.fit(train_dataset,
                epochs=10,
                validation_data=test_dataset)
```

```
Epoch 1/10
469/469             71s 144ms/step -
accuracy: 0.7979 - loss: 0.2197 - val_accuracy: 0.8139 - val_loss: 0.0791
Epoch 2/10
469/469             67s 142ms/step -
accuracy: 0.8144 - loss: 0.0775 - val_accuracy: 0.8142 - val_loss: 0.0738
Epoch 3/10
469/469             61s 131ms/step -
accuracy: 0.8147 - loss: 0.0729 - val_accuracy: 0.8144 - val_loss: 0.0715
Epoch 4/10
469/469             56s 120ms/step -
accuracy: 0.8148 - loss: 0.0707 - val_accuracy: 0.8144 - val_loss: 0.0703
Epoch 5/10
469/469             57s 122ms/step -
accuracy: 0.8149 - loss: 0.0695 - val_accuracy: 0.8144 - val_loss: 0.0692
Epoch 6/10
469/469             56s 120ms/step -
accuracy: 0.8149 - loss: 0.0687 - val_accuracy: 0.8144 - val_loss: 0.0686
Epoch 7/10
469/469             57s 122ms/step -
accuracy: 0.8149 - loss: 0.0681 - val_accuracy: 0.8144 - val_loss: 0.0677
Epoch 8/10
469/469             58s 124ms/step -
accuracy: 0.8149 - loss: 0.0676 - val_accuracy: 0.8144 - val_loss: 0.0673
Epoch 9/10
```

```
469/469             56s 119ms/step -
accuracy: 0.8149 - loss: 0.0673 - val_accuracy: 0.8144 - val_loss: 0.0670
Epoch 10/10
469/469             56s 120ms/step -
accuracy: 0.8149 - loss: 0.0669 - val_accuracy: 0.8145 - val_loss: 0.0666
```

[ ]:
```
<keras.src.callbacks.history.History at 0x7c08234d3c20>
```

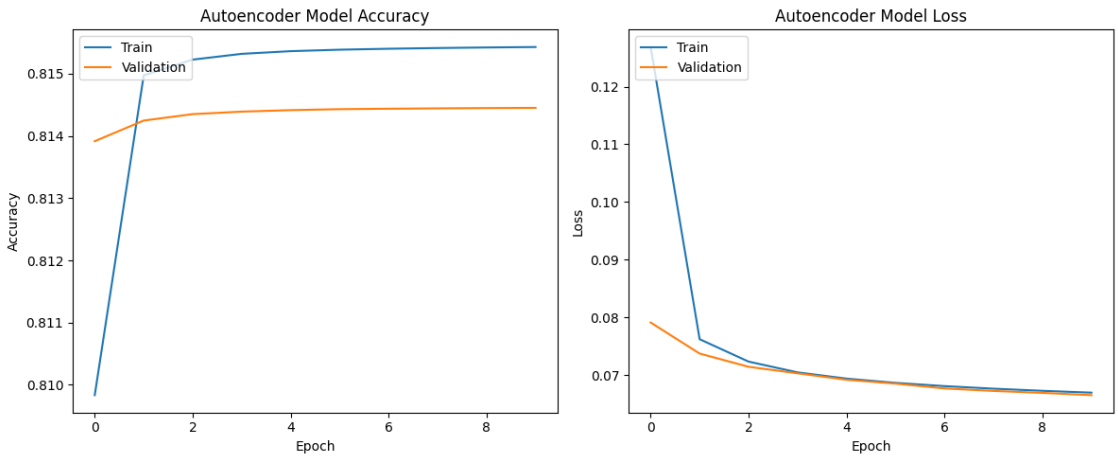
[ ]:
```
#plot the accuracy, validation accuracy and validation loss
history = autoencoder.history

plt.figure(figsize=(12, 5))

# Plot training & validation accuracy values
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Autoencoder Model Accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')

# Plot training & validation loss values
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Autoencoder Model Loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')

plt.tight_layout()
plt.show()
```



6.0.2 FEATURE EXTRACTION FROM CONVOLUTIONAL AUTO-ENCODER

[ ]:
```
encoded_features_train = autoencoder.predict(x_train)
print(f"Shape of encoded training features: {encoded_features_train.shape}")

encoded_features_test = autoencoder.predict(x_test)
print(f"Shape of encoded features: {encoded_features_test.shape}")
```

```
1875/1875             68s 36ms/step
Shape of encoded training features: (60000, 28, 28, 1)
313/313             8s 25ms/step
Shape of encoded features: (10000, 28, 28, 1)
```

[ ]:
```
flattened_features_train = encoded_features_train.
 ↪reshape(encoded_features_train.shape[0], -1)
flattened_features_test = encoded_features_test.reshape(encoded_features_test.
 ↪shape[0], -1)

print(f"Shape of flattened training features: {flattened_features_train.shape}")
print(f"Shape of flattened test features: {flattened_features_test.shape}")
```

```
Shape of flattened training features: (60000, 784)
Shape of flattened test features: (10000, 784)
```

Visualize the clusters

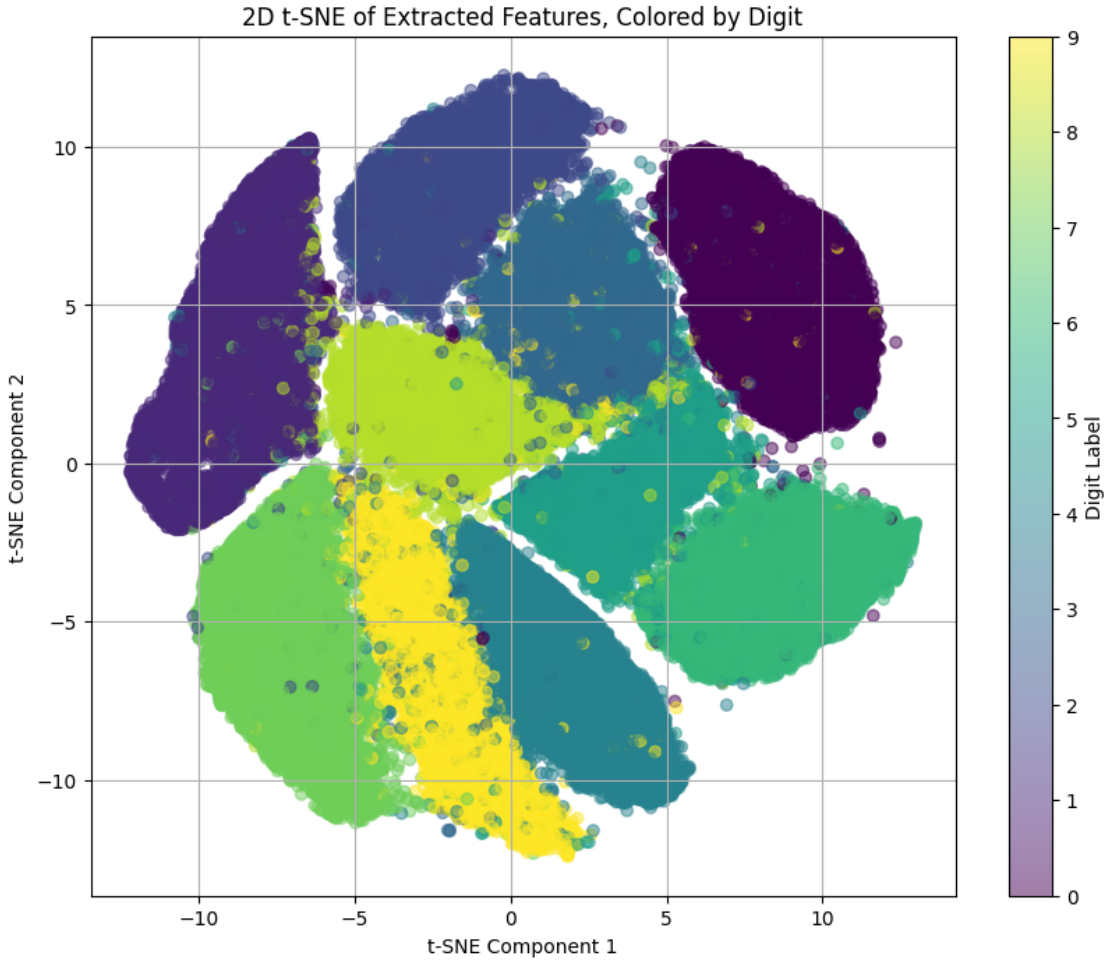
[ ]:
```
from sklearn.manifold import TSNE
tsne = TSNE(n_components=2, random_state=42, verbose=1, perplexity=30,␣
 ↪n_iter=300)
reduced_features_tsne = tsne.fit_transform(flattened_features_train)
plt.figure(figsize=(10, 8))
plt.scatter(reduced_features_tsne[:, 0], reduced_features_tsne[:, 1],␣
 ↪c=y_train, cmap='viridis', alpha=0.5)
plt.colorbar(label='Digit Label')
plt.title('2D t-SNE of Extracted Features, Colored by Digit')
plt.xlabel('t-SNE Component 1')
plt.ylabel('t-SNE Component 2')
plt.grid(True)
plt.show()
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/manifold/_t_sne.py:1164:
FutureWarning: 'n_iter' was renamed to 'max_iter' in version 1.5 and will be
removed in 1.7.
  warnings.warn(

[t-SNE] Computing 91 nearest neighbors…
```

```
[t-SNE] Indexed 60000 samples in 0.026s…
[t-SNE] Computed neighbors for 60000 samples in 183.217s…
[t-SNE] Computed conditional probabilities for sample 1000 / 60000
[t-SNE] Computed conditional probabilities for sample 2000 / 60000
[t-SNE] Computed conditional probabilities for sample 3000 / 60000
[t-SNE] Computed conditional probabilities for sample 4000 / 60000
[t-SNE] Computed conditional probabilities for sample 5000 / 60000
[t-SNE] Computed conditional probabilities for sample 6000 / 60000
[t-SNE] Computed conditional probabilities for sample 7000 / 60000
[t-SNE] Computed conditional probabilities for sample 8000 / 60000
[t-SNE] Computed conditional probabilities for sample 9000 / 60000
[t-SNE] Computed conditional probabilities for sample 10000 / 60000
[t-SNE] Computed conditional probabilities for sample 11000 / 60000
[t-SNE] Computed conditional probabilities for sample 12000 / 60000
[t-SNE] Computed conditional probabilities for sample 13000 / 60000
[t-SNE] Computed conditional probabilities for sample 14000 / 60000
[t-SNE] Computed conditional probabilities for sample 15000 / 60000
[t-SNE] Computed conditional probabilities for sample 16000 / 60000
[t-SNE] Computed conditional probabilities for sample 17000 / 60000
[t-SNE] Computed conditional probabilities for sample 18000 / 60000
[t-SNE] Computed conditional probabilities for sample 19000 / 60000
[t-SNE] Computed conditional probabilities for sample 20000 / 60000
[t-SNE] Computed conditional probabilities for sample 21000 / 60000
[t-SNE] Computed conditional probabilities for sample 22000 / 60000
[t-SNE] Computed conditional probabilities for sample 23000 / 60000
[t-SNE] Computed conditional probabilities for sample 24000 / 60000
[t-SNE] Computed conditional probabilities for sample 25000 / 60000
[t-SNE] Computed conditional probabilities for sample 26000 / 60000
[t-SNE] Computed conditional probabilities for sample 27000 / 60000
[t-SNE] Computed conditional probabilities for sample 28000 / 60000
[t-SNE] Computed conditional probabilities for sample 29000 / 60000
[t-SNE] Computed conditional probabilities for sample 30000 / 60000
[t-SNE] Computed conditional probabilities for sample 31000 / 60000
[t-SNE] Computed conditional probabilities for sample 32000 / 60000
[t-SNE] Computed conditional probabilities for sample 33000 / 60000
[t-SNE] Computed conditional probabilities for sample 34000 / 60000
[t-SNE] Computed conditional probabilities for sample 35000 / 60000
[t-SNE] Computed conditional probabilities for sample 36000 / 60000
[t-SNE] Computed conditional probabilities for sample 37000 / 60000
[t-SNE] Computed conditional probabilities for sample 38000 / 60000
[t-SNE] Computed conditional probabilities for sample 39000 / 60000
[t-SNE] Computed conditional probabilities for sample 40000 / 60000
[t-SNE] Computed conditional probabilities for sample 41000 / 60000
[t-SNE] Computed conditional probabilities for sample 42000 / 60000
[t-SNE] Computed conditional probabilities for sample 43000 / 60000
[t-SNE] Computed conditional probabilities for sample 44000 / 60000
[t-SNE] Computed conditional probabilities for sample 45000 / 60000
[t-SNE] Computed conditional probabilities for sample 46000 / 60000
```

```
[t-SNE] Computed conditional probabilities for sample 47000 / 60000
[t-SNE] Computed conditional probabilities for sample 48000 / 60000
[t-SNE] Computed conditional probabilities for sample 49000 / 60000
[t-SNE] Computed conditional probabilities for sample 50000 / 60000
[t-SNE] Computed conditional probabilities for sample 51000 / 60000
[t-SNE] Computed conditional probabilities for sample 52000 / 60000
[t-SNE] Computed conditional probabilities for sample 53000 / 60000
[t-SNE] Computed conditional probabilities for sample 54000 / 60000
[t-SNE] Computed conditional probabilities for sample 55000 / 60000
[t-SNE] Computed conditional probabilities for sample 56000 / 60000
[t-SNE] Computed conditional probabilities for sample 57000 / 60000
[t-SNE] Computed conditional probabilities for sample 58000 / 60000
[t-SNE] Computed conditional probabilities for sample 59000 / 60000
[t-SNE] Computed conditional probabilities for sample 60000 / 60000
[t-SNE] Mean sigma: 2.053728
[t-SNE] KL divergence after 250 iterations with early exaggeration: 96.087662
[t-SNE] KL divergence after 300 iterations: 4.250136
```



[ ]:
```
encoder = Model(input_img, encoded)
```

7-Segment LED Mapping

Reference:

[ ]:
```
# Define the 7-segment LED patterns for digits 0-9
# The order of segments can be A, B, C, D, E, F, G
#   A
# F   B
#   G
# E   C
#   D

seven_segment_map = {
    0: [1, 1, 1, 1, 1, 1, 0],  # All segments except G
    1: [0, 1, 1, 0, 0, 0, 0],  # B, C
    2: [1, 1, 0, 1, 1, 0, 1],  # A, B, D, E, G
    3: [1, 1, 1, 1, 0, 0, 1],  # A, B, C, D, G
    4: [0, 1, 1, 0, 0, 1, 1],  # B, C, F, G
    5: [1, 0, 1, 1, 0, 1, 1],  # A, C, D, F, G
    6: [1, 0, 1, 1, 1, 1, 1],  # A, C, D, E, F, G
    7: [1, 1, 1, 0, 0, 0, 0],  # A, B, C
    8: [1, 1, 1, 1, 1, 1, 1],  # All segments
    9: [1, 1, 1, 1, 0, 1, 1]   # A, B, C, D, F, G
}

print("7-segment LED mapping defined.")
```

```
7-segment LED mapping defined.
```

[ ]:
```
#apply 7-segment to train and test to create target variables

y_train_7segment = np.array([seven_segment_map[digit] for digit in y_train])
y_test_7segment = np.array([seven_segment_map[digit] for digit in y_test])

print(f"Shape of 7-segment training labels: {y_train_7segment.shape}")
print(f"Shape of 7-segment test labels: {y_test_7segment.shape}")
```

```
Shape of 7-segment training labels: (60000, 7)
Shape of 7-segment test labels: (10000, 7)
```

6.0.3 MLP Model for 7-LED Output

The input dimension will be the size of our flattened features, and the output layer will have 7 neurons with a sigmoid activation function

[ ]:
```
# Input dimension for the MLP is the size of the flattened features
input_dim_mlp = flattened_features_train.shape[1]

mlp_model = Sequential([
    Dense(128, activation='relu', input_shape=(input_dim_mlp,),␣
 ↪kernel_regularizer=keras.regularizers.l2(0.001)),
    Dropout(0.2),
    Dense(64, activation='relu', kernel_regularizer=keras.regularizers.l2(0.
 ↪001)),
    Dropout(0.2),
    Dense(7, activation='sigmoid') # 7 outputs for the 7 segments, sigmoid for␣
 ↪binary classification per segment
])

mlp_model.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	100,480
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 64)	8,256
dropout_1 (Dropout)	(None, 64)	0
dense_2 (Dense)	(None, 7)	455

Total params: 109,191 (426.53 KB)

Trainable params: 109,191 (426.53 KB)

Non-trainable params: 0 (0.00 B)

[ ]:
```
#binary_crossentropy is the loss function
#Adam optimizer is used

mlp_model.compile(optimizer=keras.optimizers.Adam(learning_rate=0.00001),
                  loss='binary_crossentropy',
                  metrics=['accuracy','precision','recall'])
```

[ ]:
```
mlp_model.fit(flattened_features_train, y_train_7segment,
              epochs=30,
              batch_size=32,
              validation_data=(flattened_features_test, y_test_7segment))
```

```
Epoch 1/30
1875/1875             17s 5ms/step -
accuracy: 0.1218 - loss: 0.8670 - precision: 0.7612 - recall: 0.7998 -
val_accuracy: 0.2509 - val_loss: 0.6429 - val_precision: 0.8185 - val_recall:
0.9615
Epoch 2/30
1875/1875             6s 3ms/step -
accuracy: 0.2481 - loss: 0.6378 - precision: 0.8344 - recall: 0.9273 -
val_accuracy: 0.2733 - val_loss: 0.5000 - val_precision: 0.9022 - val_recall:
0.9602
Epoch 3/30
1875/1875             7s 4ms/step -
accuracy: 0.2681 - loss: 0.5209 - precision: 0.8869 - recall: 0.9363 -
val_accuracy: 0.2992 - val_loss: 0.4199 - val_precision: 0.9265 - val_recall:
0.9640
Epoch 4/30
1875/1875             6s 3ms/step -
accuracy: 0.2811 - loss: 0.4506 - precision: 0.9111 - recall: 0.9441 -
val_accuracy: 0.3045 - val_loss: 0.3727 - val_precision: 0.9400 - val_recall:
0.9656
Epoch 5/30
1875/1875             7s 4ms/step -
accuracy: 0.2924 - loss: 0.4067 - precision: 0.9246 - recall: 0.9499 -
val_accuracy: 0.3235 - val_loss: 0.3415 - val_precision: 0.9474 - val_recall:
0.9684
Epoch 6/30
1875/1875             6s 3ms/step -
accuracy: 0.3015 - loss: 0.3762 - precision: 0.9333 - recall: 0.9537 -
val_accuracy: 0.3368 - val_loss: 0.3187 - val_precision: 0.9525 - val_recall:
0.9705
Epoch 7/30
1875/1875             7s 4ms/step -
accuracy: 0.3108 - loss: 0.3520 - precision: 0.9394 - recall: 0.9573 -
val_accuracy: 0.3463 - val_loss: 0.3005 - val_precision: 0.9563 - val_recall:
0.9721
```

```
Epoch 8/30
1875/1875            6s 3ms/step -
accuracy: 0.3189 - loss: 0.3331 - precision: 0.9447 - recall: 0.9602 -
val_accuracy: 0.3531 - val_loss: 0.2854 - val_precision: 0.9590 - val_recall:
0.9744
Epoch 9/30
1875/1875            7s 4ms/step -
accuracy: 0.3226 - loss: 0.3155 - precision: 0.9485 - recall: 0.9631 -
val_accuracy: 0.3647 - val_loss: 0.2722 - val_precision: 0.9613 - val_recall:
0.9758
Epoch 10/30
1875/1875            6s 3ms/step -
accuracy: 0.3266 - loss: 0.3014 - precision: 0.9520 - recall: 0.9651 -
val_accuracy: 0.3698 - val_loss: 0.2609 - val_precision: 0.9637 - val_recall:
0.9771
Epoch 11/30
1875/1875            7s 4ms/step -
accuracy: 0.3321 - loss: 0.2888 - precision: 0.9546 - recall: 0.9675 -
val_accuracy: 0.3717 - val_loss: 0.2507 - val_precision: 0.9660 - val_recall:
0.9779
Epoch 12/30
1875/1875            6s 3ms/step -
accuracy: 0.3368 - loss: 0.2782 - precision: 0.9566 - recall: 0.9684 -
val_accuracy: 0.3763 - val_loss: 0.2416 - val_precision: 0.9674 - val_recall:
0.9788
Epoch 13/30
1875/1875            8s 4ms/step -
accuracy: 0.3396 - loss: 0.2689 - precision: 0.9589 - recall: 0.9699 -
val_accuracy: 0.3738 - val_loss: 0.2333 - val_precision: 0.9694 - val_recall:
0.9795
Epoch 14/30
1875/1875            9s 4ms/step -
accuracy: 0.3364 - loss: 0.2587 - precision: 0.9605 - recall: 0.9717 -
val_accuracy: 0.3782 - val_loss: 0.2256 - val_precision: 0.9698 - val_recall:
0.9807
Epoch 15/30
1875/1875            6s 3ms/step -
accuracy: 0.3415 - loss: 0.2505 - precision: 0.9622 - recall: 0.9723 -
val_accuracy: 0.3796 - val_loss: 0.2186 - val_precision: 0.9710 - val_recall:
0.9815
Epoch 16/30
1875/1875            7s 4ms/step -
accuracy: 0.3441 - loss: 0.2430 - precision: 0.9632 - recall: 0.9729 -
val_accuracy: 0.3848 - val_loss: 0.2120 - val_precision: 0.9717 - val_recall:
0.9821
Epoch 17/30
1875/1875            6s 3ms/step -
accuracy: 0.3483 - loss: 0.2363 - precision: 0.9645 - recall: 0.9747 -
```

```
val_accuracy: 0.3806 - val_loss: 0.2061 - val_precision: 0.9726 - val_recall:
0.9825
Epoch 18/30
1875/1875            7s 4ms/step -
accuracy: 0.3482 - loss: 0.2293 - precision: 0.9660 - recall: 0.9752 -
val_accuracy: 0.3848 - val_loss: 0.2003 - val_precision: 0.9735 - val_recall:
0.9830
Epoch 19/30
1875/1875            10s 4ms/step -
accuracy: 0.3463 - loss: 0.2232 - precision: 0.9664 - recall: 0.9758 -
val_accuracy: 0.3901 - val_loss: 0.1952 - val_precision: 0.9739 - val_recall:
0.9836
Epoch 20/30
1875/1875            6s 3ms/step -
accuracy: 0.3527 - loss: 0.2171 - precision: 0.9678 - recall: 0.9768 -
val_accuracy: 0.3877 - val_loss: 0.1903 - val_precision: 0.9748 - val_recall:
0.9838
Epoch 21/30
1875/1875            11s 3ms/step -
accuracy: 0.3491 - loss: 0.2119 - precision: 0.9686 - recall: 0.9778 -
val_accuracy: 0.3884 - val_loss: 0.1858 - val_precision: 0.9750 - val_recall:
0.9843
Epoch 22/30
1875/1875            6s 3ms/step -
accuracy: 0.3495 - loss: 0.2063 - precision: 0.9699 - recall: 0.9783 -
val_accuracy: 0.3871 - val_loss: 0.1813 - val_precision: 0.9757 - val_recall:
0.9843
Epoch 23/30
1875/1875            8s 4ms/step -
accuracy: 0.3483 - loss: 0.2020 - precision: 0.9697 - recall: 0.9785 -
val_accuracy: 0.3830 - val_loss: 0.1774 - val_precision: 0.9763 - val_recall:
0.9843
Epoch 24/30
1875/1875            7s 4ms/step -
accuracy: 0.3499 - loss: 0.1973 - precision: 0.9706 - recall: 0.9789 -
val_accuracy: 0.3873 - val_loss: 0.1734 - val_precision: 0.9761 - val_recall:
0.9851
Epoch 25/30
1875/1875            6s 3ms/step -
accuracy: 0.3485 - loss: 0.1932 - precision: 0.9716 - recall: 0.9799 -
val_accuracy: 0.3916 - val_loss: 0.1700 - val_precision: 0.9772 - val_recall:
0.9848
Epoch 26/30
1875/1875            7s 4ms/step -
accuracy: 0.3531 - loss: 0.1897 - precision: 0.9721 - recall: 0.9799 -
val_accuracy: 0.3913 - val_loss: 0.1665 - val_precision: 0.9771 - val_recall:
0.9853
Epoch 27/30
```

```
1875/1875              6s 3ms/step -
accuracy: 0.3539 - loss: 0.1851 - precision: 0.9726 - recall: 0.9807 -
val_accuracy: 0.3878 - val_loss: 0.1633 - val_precision: 0.9779 - val_recall:
0.9854
Epoch 28/30
1875/1875              7s 4ms/step -
accuracy: 0.3527 - loss: 0.1821 - precision: 0.9731 - recall: 0.9806 -
val_accuracy: 0.3889 - val_loss: 0.1603 - val_precision: 0.9781 - val_recall:
0.9856
Epoch 29/30
1875/1875              7s 3ms/step -
accuracy: 0.3543 - loss: 0.1796 - precision: 0.9733 - recall: 0.9810 -
val_accuracy: 0.3851 - val_loss: 0.1575 - val_precision: 0.9783 - val_recall:
0.9861
Epoch 30/30
1875/1875              7s 4ms/step -
accuracy: 0.3522 - loss: 0.1760 - precision: 0.9739 - recall: 0.9811 -
val_accuracy: 0.3879 - val_loss: 0.1548 - val_precision: 0.9785 - val_recall:
0.9864
```

[ ]:
```
<keras.src.callbacks.history.History at 0x7c082279d970>
```

[ ]:
```
#predict using MLP classifier
y_pred_mlp = mlp_model.predict(flattened_features_test)
```

```
313/313              2s 4ms/step
```

[ ]:
```
#calculate accuracy on the test dataset of the MLP classifier
from sklearn.metrics import accuracy_score
accuracy_mlp = accuracy_score(y_test_7segment, y_pred_mlp.round())
print(f"Accuracy of MLP Classifier: {accuracy_mlp}")
```

```
Accuracy of MLP Classifier: 0.903
```

Visualize Predicted 7-Segment LED Displays

[ ]:
```
# Get MLP predictions for the test set
y_pred_mlp_prob = mlp_model.predict(flattened_features_test)
# Convert probabilities to binary (0 or 1) using a threshold of 0.5
y_pred_mlp_binary = (y_pred_mlp_prob > 0.5).astype(int)

# Function to draw a 7-segment display
def draw_7_segment(ax, segments, title=""):
    # Segment coordinates (relative to a 10x10 grid for drawing)
    #   A
    # F   B
    #   G
    # E   C
```

```
    #   D
    segment_coords = {
        0: [(2, 9), (8, 9)], # A
        1: [(9, 8), (9, 2)], # B
        2: [(8, 1), (2, 1)], # C
        3: [(1, 2), (1, 8)], # D
        4: [(2, 0), (8, 0)], # E
        5: [(0, 8), (0, 2)], # F
        6: [(2, 5), (8, 5)]  # G
    }

    ax.set_xlim(-1, 11)
    ax.set_ylim(-1, 11)
    ax.set_aspect('equal', adjustable='box')
    ax.axis('off')
    ax.set_title(title)

    for i, (start, end) in segment_coords.items():
        color = 'red' if segments[i] == 1 else 'lightgray'
        # Adjust thickness for horizontal vs vertical
        if i in [0, 2, 4, 6]: # Horizontal segments
            ax.add_patch(patches.Rectangle((start[0], start[1] - 0.5), end[0] -␣
 ↪start[0], 1, color=color))
        else: # Vertical segments
            ax.add_patch(patches.Rectangle((start[0] - 0.5, start[1]), 1,␣
 ↪end[1] - start[1], color=color))

# Display predictions for a few examples of each digit
num_examples_per_digit = 3

plt.figure(figsize=(15, 2 * 10 * num_examples_per_digit))

for digit in range(10):
    # Get indices for this digit from the test set
    digit_indices = np.where(y_test == digit)[0]
    np.random.shuffle(digit_indices) # Shuffle to get random examples

    for i in range(min(num_examples_per_digit, len(digit_indices))):
        idx = digit_indices[i]

        # Original MNIST Image
        ax1 = plt.subplot(10 * num_examples_per_digit, 3, (digit *␣
 ↪num_examples_per_digit + i) * 3 + 1)
        ax1.imshow(x_test[idx].reshape(28, 28), cmap='gray')
        ax1.set_title(f"Original ({digit})")
        ax1.axis('off')
```

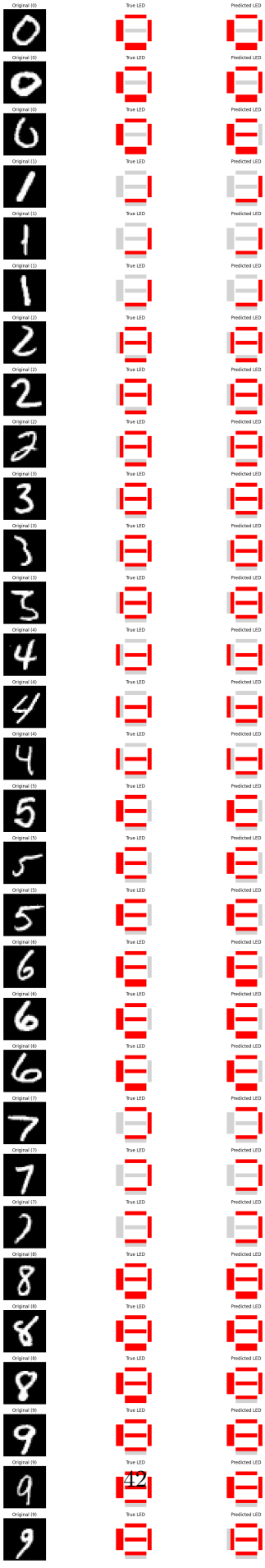
```
        # True 7-segment LED
        ax2 = plt.subplot(10 * num_examples_per_digit, 3, (digit *␣
↪num_examples_per_digit + i) * 3 + 2)
        draw_7_segment(ax2, y_test_7segment[idx], "True LED")

        # Predicted 7-segment LED
        ax3 = plt.subplot(10 * num_examples_per_digit, 3, (digit *␣
↪num_examples_per_digit + i) * 3 + 3)
        draw_7_segment(ax3, y_pred_mlp_binary[idx], "Predicted LED")

plt.tight_layout()
plt.show()

print("Visualization of predicted 7-segment LED displays complete.")
```

```
313/313             1s 3ms/step
```


Original (0)
True LED
Predicted LED
Original (0)
True LED
Predicted LED
Original (0)
True LED
Predicted LED
Original (1)
True LED
Predicted LED
Original (1)
True LED
Predicted LED
Original (1)
True LED
Predicted LED
Original (2)
True LED
Predicted LED
Original (2)
True LED
Predicted LED
Original (2)
True LED
Predicted LED
Original (3)
True LED
Predicted LED
Original (3)
True LED
Predicted LED
Original (3)
True LED
Predicted LED
Original (4)
True LED
Predicted LED
Original (4)
True LED
Predicted LED
Original (4)
True LED
Predicted LED
Original (5)
True LED
Predicted LED
Original (5)
True LED
Predicted LED
Original (5)
True LED
Predicted LED
Original (6)
True LED
Predicted LED
Original (6)
True LED
Predicted LED
Original (6)
True LED
Predicted LED
Original (7)
True LED
Predicted LED
Original (7)
True LED
Predicted LED
Original (7)
True LED
Predicted LED
Original (8)
True LED
Predicted LED
Original (8)
True LED
Predicted LED
Original (8)
True LED
Predicted LED
Original (9)
True LED
Predicted LED
Original (9)
True LED
Predicted LED
Original (9)
True LED
Predicted LED

Visualization of predicted 7-segment LED displays complete.

[ ]: